



Título: Reporte del Proyecto Snake Game

Estudiante: Daniel Canales Taylor

Carrera: Ingeniería de Software, UIDE

Asignatura: Lógica de Programación

Docente: Ing. Estefania Heredia Jimenez

Fecha: 20 de Agosto del 2026

Introducción

El presente proyecto consiste en el desarrollo de un juego clásico de la serpiente (Snake) utilizando el lenguaje de programación Python y la librería Pygame. El Snake es uno de los videojuegos más reconocidos en la historia de la programación, ya que combina lógica básica, manejo de estructuras de datos y control de eventos, lo que lo convierte en un ejercicio ideal para aplicar los conocimientos adquiridos en la materia de Lógica de Programación.

El objetivo del proyecto es aplicar conceptos fundamentales de programación variables, listas, tuplas, strings, funciones, estructuras de control, condicionales y bucles en un caso práctico e interactivo. Adicionalmente, como decisión propia del desarrollo, se incorporó programación orientada a objetos para organizar el código en módulos más claros y reutilizables.

Descripción del problema

El problema consiste en crear un videojuego donde el jugador controla una serpiente que se mueve por una cuadrícula rectangular (grid) y debe consumir alimentos que aparecen aleatoriamente en la pantalla. Cada alimento ingerido aumenta el tamaño de la serpiente y otorga 10 puntos de puntuación. El juego finaliza cuando la serpiente colisiona con las paredes del área de juego o con su propio cuerpo.

Los desafíos técnicos identificados fueron:

- Representar el estado del juego de forma continua usando un bucle principal.
- Controlar el movimiento de la serpiente de manera fluida y con límites lógicos (no permitir giros de 180 grados).
- Generar la comida en posiciones aleatorias sin superponerla con el cuerpo de la serpiente.
- Detectar colisiones de forma precisa (paredes, alimento, cuerpo propio).
- Guardar y recuperar las mejores puntuaciones entre sesiones de ejecución.
- Diseñar una interfaz gráfica clara y funcional con múltiples pantallas.

Relación con los contenidos de la asignatura

El proyecto aplica los conceptos fundamentales vistos en la asignatura de Lógica de Programación:

a) Variables:

- Se utilizan variables para almacenar el estado del juego: puntaje actual, dirección de movimiento, estado actual del programa (inicio, jugando, entrada de nombre), tamaño de la ventana, tamaño de las celdas, etc.
- Ejemplo: la variable "score" almacena la puntuación y se incrementa en 10 cada vez que la serpiente consume un alimento.

b) Strings y sus métodos:

- Los strings se usan para el título del juego ("SNAKE"), mensajes en pantalla ("FIN DEL JUEGO", "Puntuacion:"), nombres de archivos ("scores.json") y para construir el texto que se muestra al jugador.
- Se aplican métodos de strings como upper() para convertir el nombre del

jugador a mayúsculas, `str()` para convertir números a texto al mostrar la puntuación, y operaciones de concatenación para formar mensajes dinámicos como "Puntuacion: 50".

c) Listas y sus métodos:

- La lista es la estructura de datos principal del juego. El cuerpo de la serpiente se representa como una lista de tuplas, donde cada tupla es una coordenada (columna, fila).
- Se usan métodos de listas como `append()` para agregar un nuevo segmento al crecer, `pop()` para eliminar la cola al moverse, `insert(0, ...)` para agregar la nueva cabeza al inicio, y `"in"` para verificar si una posición está ocupada (colisión con el cuerpo propio o generación de comida).
- Las puntuaciones también se almacenan como una lista de diccionarios.

d) Tuplas:

- Las tuplas se usan para representar posiciones y direcciones de la serpiente en el grid. Cada posición es una tupla (columna, fila) como (5, 8). Las direcciones son tuplas como (1, 0) para derecha, (-1, 0) para izquierda, (0, 1) para abajo y (0, -1) para arriba.
- Las tuplas son ideales porque son inmutables: una vez definida una posición o dirección, no cambia accidentalmente.

e) Diccionarios:

- Las puntuaciones se guardan como diccionarios con las claves "name" y "score", por ejemplo: `{"name": "Daniel", "score": 50}`.
- Esto permite organizar la información de cada jugador de forma clara.

f) Funciones:

- El código se organiza en funciones y métodos que realizan tareas específicas. Cada función tiene un propósito claro: dibujar la serpiente, mover la serpiente, verificar colisiones, generar comida, cargar y guardar puntuaciones, dibujar pantallas, etc.
- Las funciones permiten reutilizar código y mantener el programa ordenado.

g) Estructuras de control:

- Condicionales (if/elif/else): se usan para verificar colisiones (si la cabeza toca una pared, si toca su propio cuerpo, si toca la comida), para controlar el estado del programa (si está en pantalla de inicio, jugando o entrada de nombre), y para procesar las teclas presionadas por el jugador.
- Bucles (while): el bucle principal del juego se ejecuta continuamente mientras el programa está abierto, procesando eventos, actualizando el estado y dibujando la pantalla en cada iteración. También se usa un bucle while para generar la posición de la comida hasta que sea válida (que no esté sobre la serpiente).
- Bucles (for): se usan para recorrer el cuerpo de la serpiente y dibujar cada segmento, para recorrer la lista de puntuaciones y mostrarlas en pantalla, y para dibujar las líneas de la cuadrícula.

h) Manejo de eventos del teclado:

- En cada iteración del bucle principal se capturan los eventos de teclado.
- Las teclas de flecha controlan la dirección de la serpiente: si el jugador presiona flecha arriba, la serpiente cambia su dirección a (0, -1); flecha abajo cambia a (0, 1), y así sucesivamente.
- El sistema rechaza giros de 180 grados: si la serpiente va hacia la derecha, no puede girar instantáneamente hacia la izquierda.
- En la pantalla de entrada de nombre, las teclas de letras y números se capturan para construir el nombre del jugador, con un límite de 5 caracteres. La tecla Borrar (Backspace) elimina el último carácter. La tecla Enter confirma el nombre. La tecla Esc cierra el programa.

i) Persistencia de datos:

- Se leen y escriben archivos JSON para guardar las mejores puntuaciones entre ejecuciones del programa. Se usa la librería json de Python para convertir entre listas de diccionarios y texto en el archivo.

j) Programación orientada a objetos (decisión propia):

- Aunque no fue un tema visto en clase, se decidió incorporar programación orientada a objetos para organizar el código. Se crearon clases como Snake, Food, Game, StartScreen y NameInput, cada una con sus atributos y métodos.
- Esto permite que cada componente del juego esté encapsulado en su propio módulo, facilitando la lectura y el mantenimiento del código.

Explicación del sistema Desarrollado

El sistema fue desarrollado en Python 3.12 usando la librería Pygame para la renderización gráfica. El gestor de dependencias utilizado es uv.

ESTRUCTURA DEL PROYECTO:

src/

snake_game/

__init__.py Punto de entrada (re-exporta main)

main.py Inicialización de Pygame y máquina de estados

core/

game.py Orquestador del juego (actualización, dibujado, colisiones)

snake.py Movimiento, crecimiento, cambio de dirección, dibujado de la serpiente

food.py Generación de comida (evita el cuerpo de la serpiente) y su dibujado

start_screen.py Pantalla de inicio con tabla de puntuaciones y botón JUGAR

name_input.py Pantalla de entrada de nombre después de finalizar la partida

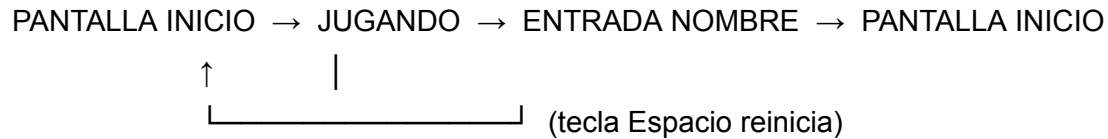
utils/

constants.py Constantes del juego (tamaño de ventana, colores, tamaño del grid, estado inicial)

score_manager.py Carga y guardado de puntuaciones en scores.json

MÁQUINA DE ESTADOS:

El programa utiliza una máquina de tres estados:



Estado 1 - Pantalla de Inicio:

Se muestra una tabla con las 10 mejores puntuaciones almacenadas, el título "SNAKE" y un botón "JUGAR". Al hacer clic en el botón o presionar la tecla Espacio, el juego comienza.

Estado 2 - Jugando:

La serpiente se mueve automáticamente hacia la derecha. El jugador la controla con las teclas de flecha. La comida aparece en posiciones aleatorias. Cada alimento consumido suma 10 puntos. El juego termina al colisionar con una pared o con el propio cuerpo.

Estado 3 - Entrada de Nombre:

Se muestra la puntuación alcanzada y un campo de texto donde el jugador ingresa su nombre (máximo 5 caracteres). La tecla Enter guarda la puntuación y regresa a la pantalla de inicio. La tecla Espacio inicia una nueva partida. La tecla Esc cierra el juego.

PANTALLA DE JUEGO:

- Cuadrícula de 30x30 celdas, cada una de 33x33 píxeles.
- Ventana total de 990x990 píxeles.
- Las 3 filas superiores están reservadas para el título y la puntuación.
- Velocidad de actualización: 7 cuadros por segundo (FPS).
- La serpiente inicia en la posición (5,8) con 3 segmentos moviéndose hacia a la derecha.
- El alimento se dibuja como un círculo rojo en el centro de la celda.

- La serpiente se dibuja como rectángulos verdes; la cabeza en verde oscuro.

COLISIONES:

1. Con alimento: la serpiente crece, se suma puntaje, la comida reaparece.
2. Con paredes: si la cabeza sale del rango $[0,29]$ en X o de $[3,29]$ en Y, termina el juego.
3. Con el cuerpo propio: si la cabeza coincide con cualquier otro segmento del cuerpo, termina el juego.

PERSISTENCIA:

Las puntuaciones se almacenan en un archivo `scores.json` en la raíz del proyecto. El sistema guarda las 10 mejores puntuaciones ordenadas de mayor a menor. El archivo se crea automáticamente en la primera ejecución.

Reflexión sobre el impacto de la tecnología

El desarrollo de este proyecto permite comprender de manera práctica cómo las tecnologías de programación transforman ideas en soluciones funcionales. A través de la creación del Snake Game, se evidencia que la programación no es solo escribir código, sino resolver problemas de forma estructurada y lógica.

La tecnología impacta en diversas dimensiones:

APRENDIZAJE Y EDUCACIÓN:

La programación es una herramienta fundamental para el desarrollo del pensamiento analítico y la resolución de problemas. Proyectos como este demuestran que la tecnología educativa puede hacer tangible y entretenido el aprendizaje de conceptos abstractos como las estructuras de datos, la lógica condicional y la gestión de estados.

ACCESIBILIDAD:

Herramientas como Python y Pygame son de código abierto y están disponibles para cualquier persona con acceso a una computadora. Esto democratiza el acceso al desarrollo de software, permitiendo que estudiantes y profesionales creen soluciones sin barreras económicas significativas.

INNOVACIÓN CREATIVA:

La tecnología permite transformar conceptos simples en experiencias interactivas. Un juego como el Snake, aparentemente sencillo, involucra múltiples áreas del conocimiento: matemáticas (coordenadas, aleatoriedad), diseño de interfaces, ingeniería de software y persistencia de datos.

IMPACTO PROFESIONAL:

Las habilidades adquiridas en este proyecto — manejo de variables, listas, tuplas, funciones, bucles, condicionales y archivos — son la base sobre la cual se construyen aplicaciones más complejas. Estos conceptos son directamente transferibles al desarrollo profesional de software, aplicaciones móviles, videojuegos y sistemas interactivos.

CONCLUSIÓN:

La tecnología, bien utilizada, es un medio poderoso para la creación, el aprendizaje y la innovación. Este proyecto refuerza la idea de que la programación es una competencia esencial en el mundo actual, y que su dominio abre puertas a la resolución de problemas en cualquier disciplina.